

课程须知

- 本课程适用软件版本：MWORKS.Syslab2024a
- 本课程示例运行需要软件首选项加载：基础库、数学库、图形库、统计库、优化库、控制库、信号处理库

MWORKS.Syslab信号处理工具箱应用

陆瑞琨

苏州同元软控信息技术有限公司

2025年5月23日



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. 功能概况

2. 信号预处理

3. 特征提取

4. 滤波器设计与分析

5. 频谱分析

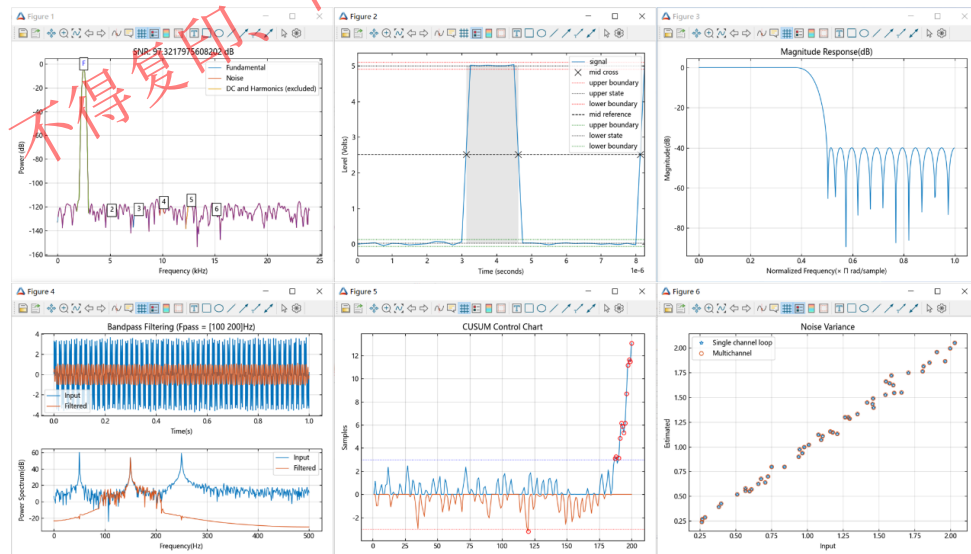
6. 综合示例

1.1 功能概述

信号处理，就是抽取出有用信息的过程，它是对信号进行提取、变换、分析、综合等处理过程的统称。

产品功能

- **信号预处理**：对信号进行去噪、平滑和去趋势处理，为进一步分析做好准备。
- **特征提取和测量**：测量和提取信号中的独特特征，包括峰值、功率、带宽、失真和信号统计信息，同时支持计算与脉冲和瞬态相关的指标。
- **滤波器设计和分析**：设计、分析和实现数字和模拟滤波器，支持使用滤波器设计应用程序或函数来设计各种数字 FIR 和 IIR 滤波器，如低通、高通和带阻滤波器。
- **频谱分析**：使用频谱估计和子空间方法表征信号的频率成分，以及设计和分析数据窗。



MWORKS信号处理工具箱旨在为信号在时、频域处理与分析提供各种常用算法和应用程序

1.1 功能概述

1 信号分析和可视化

- 信号可视化
- 数据存储和数据导入

2 信号生成和预处理

- 平滑去噪
- 波形生成

3 测量和特征提取

- 描述性统计量
- 脉冲和瞬态指标
- 频谱测量

4 变换、相关性和建模

- 变换
- 相关性和卷积
- 信号建模

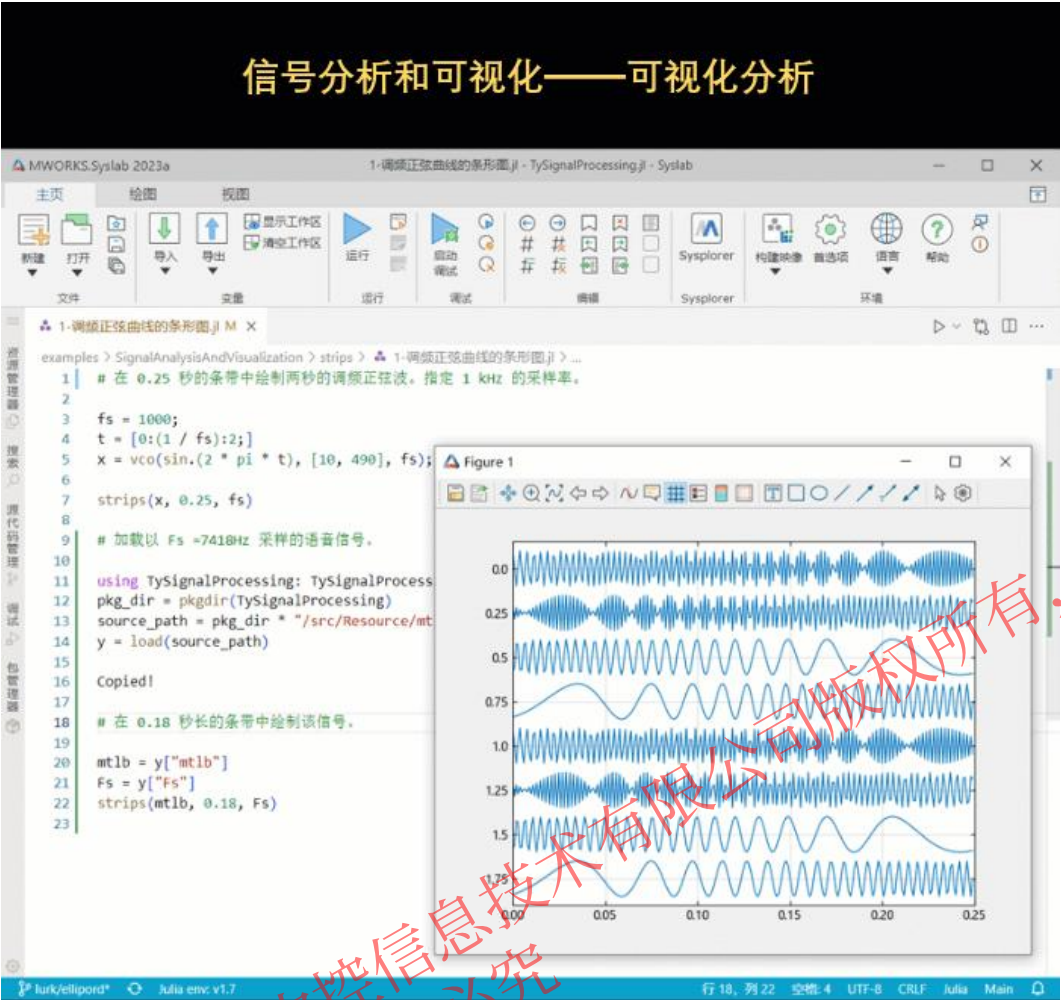
5 数字和模拟滤波器

- 数字滤波器设计
- 数字滤波器分析
- 数字滤波
- 多采样率信号处理
- 模拟滤波器

6 频谱分析

- 频谱估计
- 参数化频谱估计
- 子空间法
- 加窗法

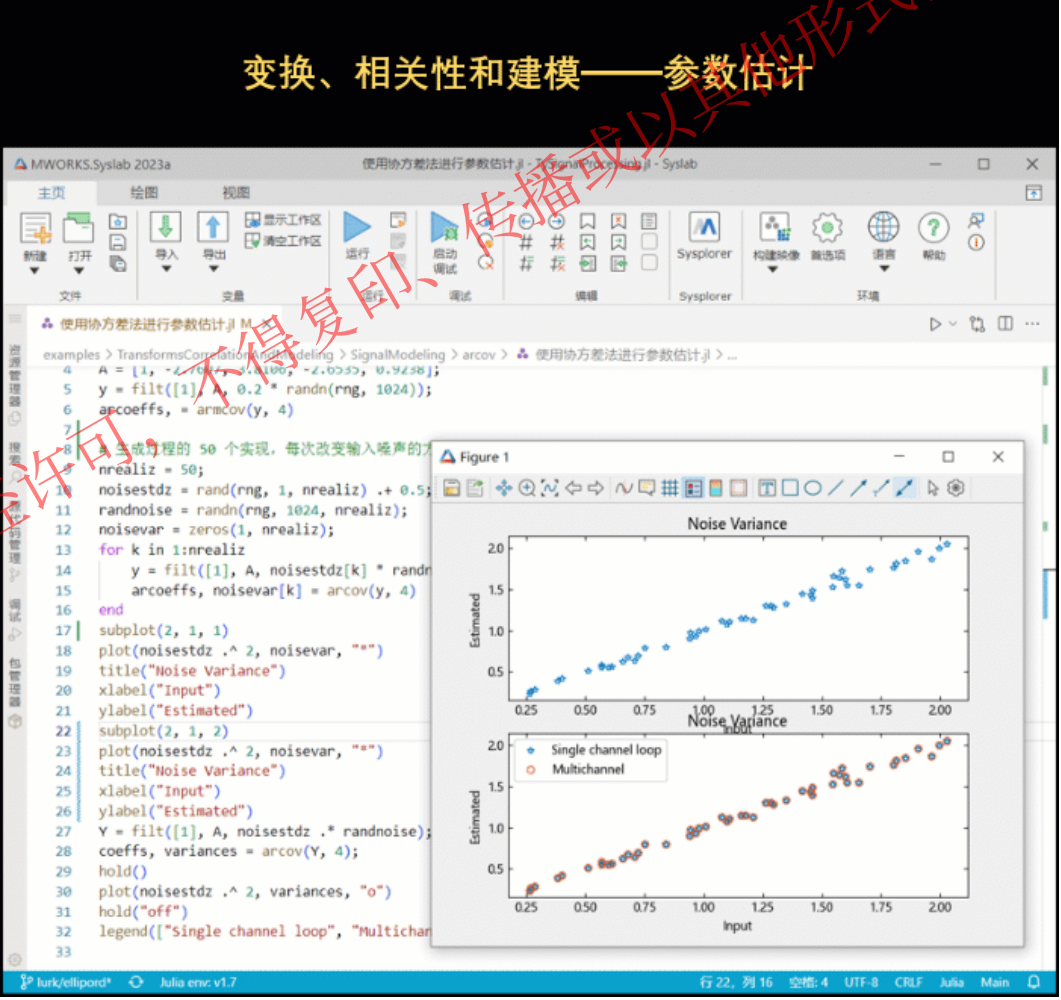
1.2 典型应用



信号处理工具箱示例1-3.gif

实例见:

- EXAMPLE\01-SignalProcessing\01-信号分析和可视化——可视化分析.jl
- EXAMPLE\01-SignalProcessing\ 02-信号生成和预处理——中值滤波降噪.jl
- EXAMPLE\01-SignalProcessing\ 03-测量和特征提取——脉宽估计



信号处理工具箱示例4-6.gif

实例见:

- EXAMPLE\01-SignalProcessing\04-变换、相关性和建模——参数估计.jl
- EXAMPLE\01-SignalProcessing\ 05-数字和模拟滤波器——模拟IIR滤波器设计
- EXAMPLE\01-SignalProcessing\ 06-频谱分析——自回归功率谱密度估计

目录

1. 功能概况

2. 信号预处理

3. 特征提取

4. 滤波器设计与分析

5. 频谱分析

6. 综合示例

2.1 信号预处理 – 信号平滑

从现实环境采集到的数据中经常混叠有微弱噪声，其中包括由于系统不稳定产生的噪声，也有周围环境引入的毛刺，这些弱噪声都需要在处理信号之前尽可能地消除或减弱，这一工作往往作为信号的预处理。

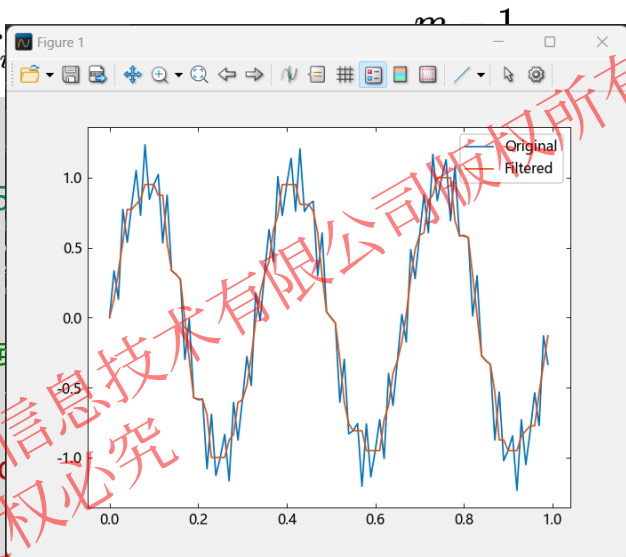
一. 平滑滤波

平滑是去掉信号中没必要的快速起伏的成份，让信号看起来更光滑，本质上就是去掉高频分量。通过平滑处理，我们可以发现数据中的重要模式，同时忽略不重要的内容（如噪声），通常使用滤波来执行这种平滑处理。

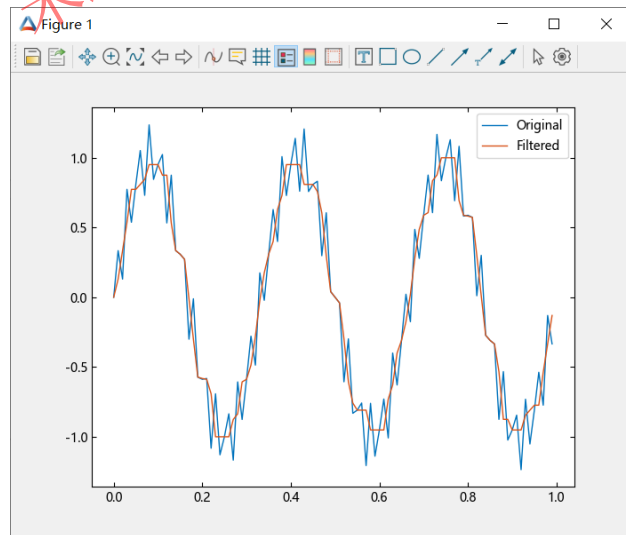
示例1：基于中值滤波的信号平滑

$$y_i = \text{Med}\{f_{i-v}, \dots, f_i, \dots, f_{i+v}\}$$

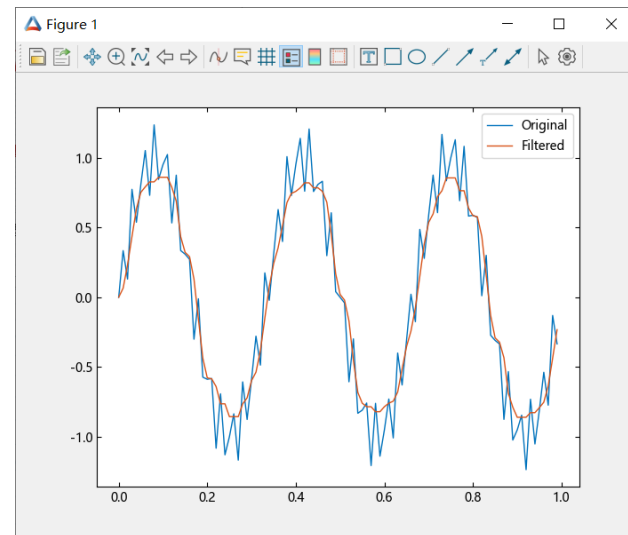
```
fs = 100;  
t = 0:(1 / fs):(1 - 1 / fs);  
x = sin.(2 * pi * t * 3) + 0.25 * randn(1, length(t));  
# 引入噪声  
y = medfilt1(x, 5);  
y1 = medfilt1(x, 10);  
# 中值滤波，指定窗长度为5  
figure(1)  
plot(t, x, t, y)  
legend(['Original', 'Filtered'])  
figure(2)  
plot(t, x, t, y1)  
legend(['Original', 'Filtered'])
```



窗长度为5阶的中值滤波



窗长度为10阶的中值滤波



阶数越高越平滑

2.1 信号预处理 – 信号平滑

示例2：基于SG滤波的信号平滑

Savitzky-Golay滤波器最初由**Savitzky**和**Golay**于1964年提出，之后被广泛地运用于数据流平滑除噪，是一种在时域内基于**局域多项式最小二乘法拟合**的**滤波方法**。

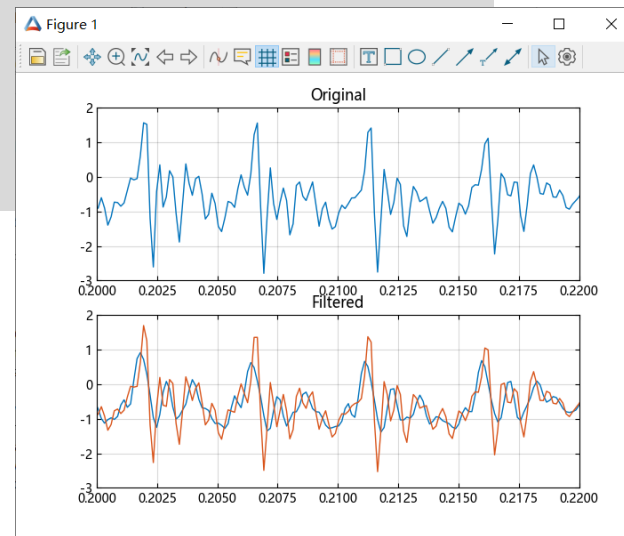
- 1、指定滤波器宽度为 $n = 2m + 1$ ，各测试点 $x = (-m, -m + 1, \dots, 0, \dots, m - 1, m)$ 采用 $k - 1$ 次多项式对数据进行拟合；
- 2、 N 个方程构成 k 元线性方程组，通过最小二乘法求解方程组系数。

$$y = a_0 + a_1x + a_2x^2 + \dots + a_{k-1}x^{k-1}$$

$$\begin{pmatrix} y_{-m} \\ y_{-m+1} \\ \vdots \\ y_m \end{pmatrix} = \begin{pmatrix} 1 & -m & \cdots & (-m)^{k-1} \\ 1 & -m+1 & \cdots & (-m+1)^{k-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & m & \cdots & m^{k-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{k-1} \end{pmatrix} + \begin{pmatrix} e_{-m} \\ e_{-m+1} \\ \vdots \\ e_m \end{pmatrix}$$

```
using TySignalProcessing: TySignalProcessing
pkg_dir = pkgdir(TySignalProcessing)
source_path = pkg_dir *
"/examples/SignalGenerationAndPreprocessing/SmoothingAnd
Denoising/sgolayfilt/data_sgolayfilt.jl"
include(source_path)
t = (0:(length(mtlb) - 1)) / Fs;
rd = 9;
fl = 21;
smtlb = sgolayfilt(mtlb, rd, fl);
# SG滤波
kmtlb = sgolayfilt(mtlb, rd, fl, kaiser(fl, 38));
# 指定凯撒窗为权重向量的SG滤波
subplot(2, 1, 1)
plot(t, mtlb);
axis([0.2 0.22 -3 2]);title("Original");grid()
subplot(2, 1, 2)
plot(t, smtlb);hold("on");
title("Filtered");grid()
plot(t, kmtlb);
axis([0.2 0.22 -3 2]);
hold("off")
```

不同权重下的SG滤波对比



2.1 信号预处理 – 信号平滑

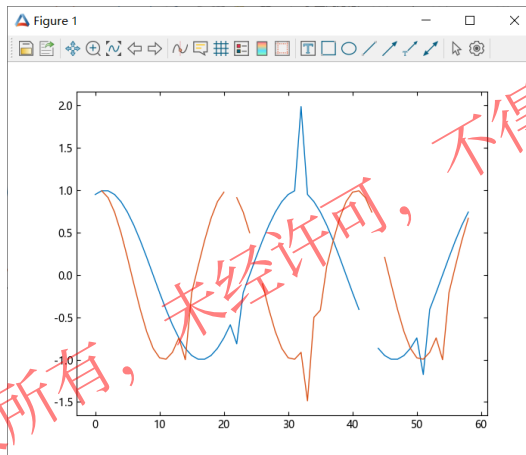
二. 信号修复

时间信号采集过程中，有时候会因为各种原因导致部分采样点的样本缺失，这里通过使用 NAN 随机添加缺失样本。通过信号预处理方法就可快速的完成信号缺失点修复

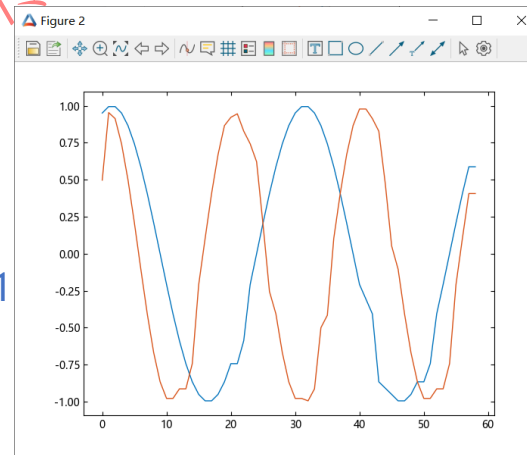
示例3：基于中值滤波的缺失点修复

```
rng = MT19937ar(1234)
n = 59
x = sin.(pi ./ [15 10]' * transpose((1:n)[:]) .+ pi / 3)'
spk = rand(rng, 1:(2 * n), 9, 1)
x[spk] = x[spk] * 2
x[rand(rng, 1:(2 * n), 6, 1)] .= NaN
# 随机注入异常值
y1 = medfilt1(x', [], [], 2;
nanflag="omitnan")
# 忽略异常值，端点外为零补全
y2 = medfilt1(x, 4, [], [];
nanflag="omitnan", padding="truncate");
# 忽略异常值，端点外为截断
figure(1)
plot(x)
figure(2)
plot(y1')
figure(3)
plot(y2)
```

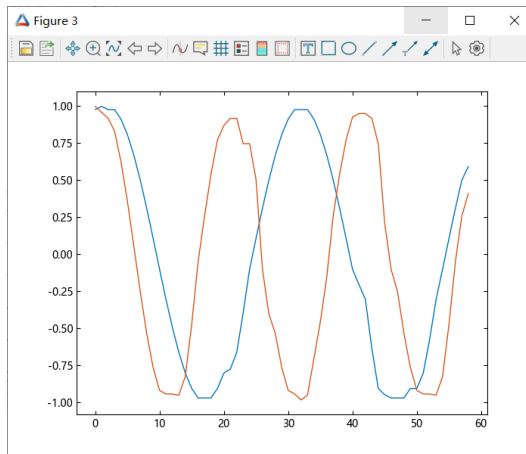
原始信号



信号修复1



信号修复2



关键字参数设置

nanflag = includenan | omitnan

includenan: 保留nan

omitnan: 忽略nan

padding = zeropad | truncate

zeropad(default): 端点外为零

truncate: 当它到达信号边缘时，计算较小段的中值。

2.1 信号预处理 – 信号平滑

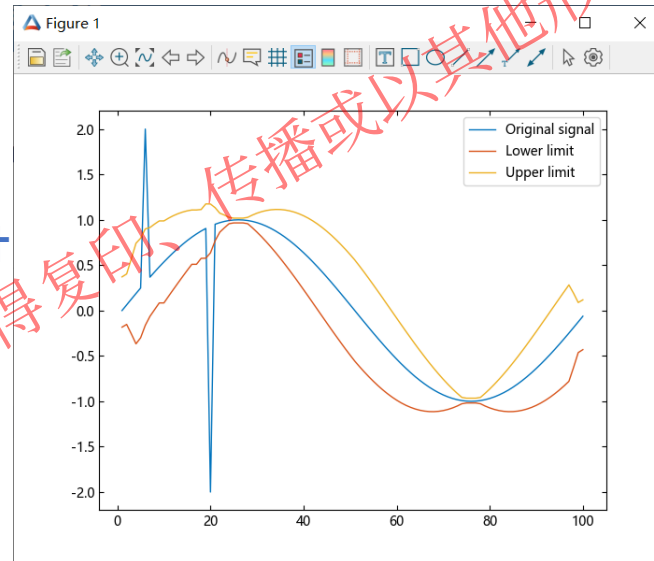
示例4：使用Hampel滤波器去标识异常值，计算局部中值、估计标准差

使用Hampel滤波器去标识异常值，计算局部中值、估计标准差

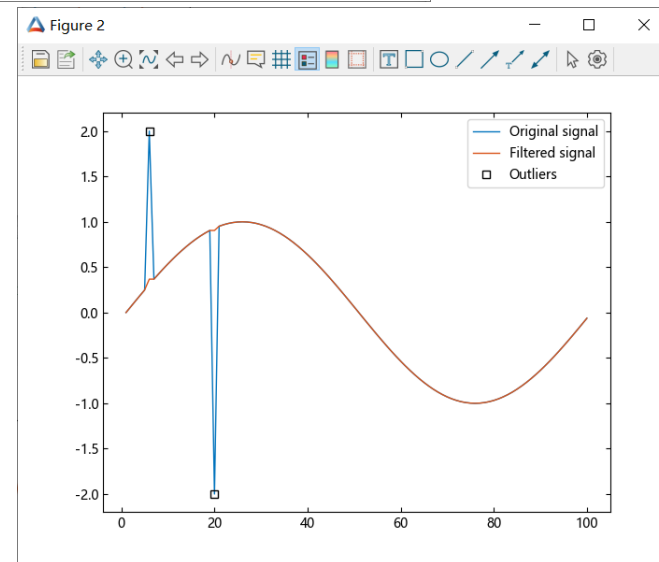
```
x = sin.(2 * pi * [0:99] / 100)
x[6] = 2
x[20] = -2
# 随机注入异常值
y, i, xmedian, xsigma = hampel(x)
# 计算局部中值、估计标准差
n = [1:length(x);]
figure(1)
plot(n, x)
hold("on")
plot(n, xmedian - 3 * xsigma, n, xmedian + 3 * xsigma)
# 绘制估计范围界限，包括上限和下限
legend(["Original signal", "Lower limit", "Upper limit"])
figure(2)
plot(n, x)
hold("on")
plot(n, y)
idx = findall(x -> x > 0, i)
# 获取异常值下标
plot(n[idx], x[idx], "sk")
legend(["Original signal", "Filtered signal", "Outliers"])
```

基于中值、标准差估计
的数据点区间范围

上限：Upper limit
下限：Lower limit



标注移除点并修复
异常值：Outliers



2.2 信号预处理 – 辅助波形生成

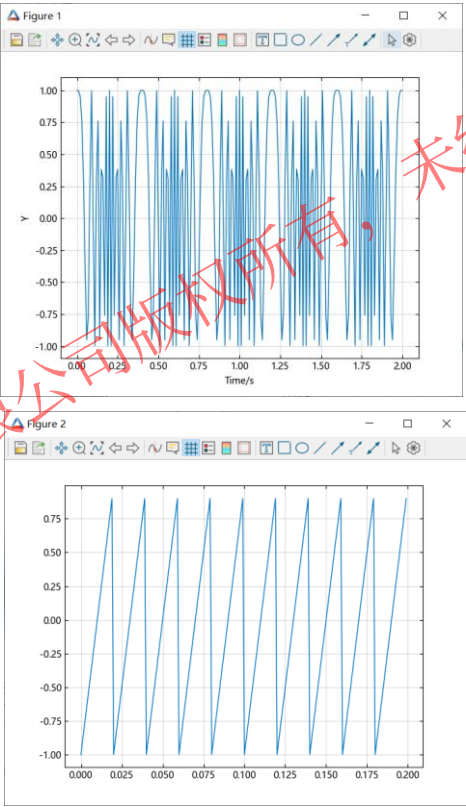
三.辅助波形生成

信号处理库提供脉冲、chirp、VCO、正弦函数、周期性/非周期性和调制信号等常用波形，可使用chirp生成线性、二次和对数chirp。使用square、rectpuls和sawtooth创建方波、矩形波和三角形波等。

示例5：扫频余弦及锯齿波生成

```
t = 0:0.01:2
y = chirp(collect(t), 0, 1, 250)
# 扫频余弦信号
figure(1)
plot(t, y)
xlabel("Time/s")
ylabel("Y")
grid("on")
```

```
T = 10 * (1 / 50)
fs = 1000
t = 0:1/fs:T-1/fs
y = sawtooth(2 * pi * 50 * t)
# 锯齿波信号
figure(2)
plot(t, y)
grid("on")
```



常用波形/序列生成函数	说明
chirp	扫频余弦
cw	cw调制信号
diric	Dirichlet或周期Sinc函数
gauspuls	高斯调制正弦射频脉冲
gmonopuls	高斯单脉冲
gmseq	广义m序列
mseq	m序列
pulstran	脉冲序列
rectpuls	采样的非周期性矩形
sawtooth	锯齿波或三角波
sinc	Sinc函数
square	方波
tripuls	采样的非周期性三角形
vco	压控振荡器

目录

1. 功能概况

2. 信号预处理

3. 特征提取

4. 滤波器设计与分析

5. 频谱分析

6. 综合示例

3.1 信号特征提取 – 时域特征

信号常见特征提取内容包括时域和频域特征，其中：

- **时域特征**：如定位信号波峰并确定其高度、宽度和与邻点的距离，如峰间幅值和信号包络，测量脉冲指标，如过冲和占空比。
- **频域特征**：如测量基频、均值频率、中位数频率和谐波频率、通道带宽和频带功率。通过测量无乱真动态范围 (SFDR)、信噪比 (SNR)、总谐波失真 (THD)、信号与噪声失真比 (SINAD) 和三阶截距 (TOI) 来表征系统。

一. 描述性统计量特征

示例1：统计序列周期

用于识别序列是否为周期序列，并输出序列每个周期长度

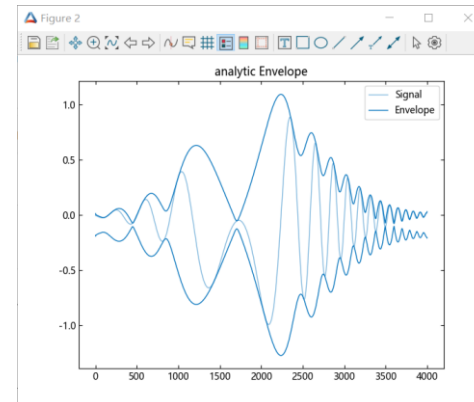
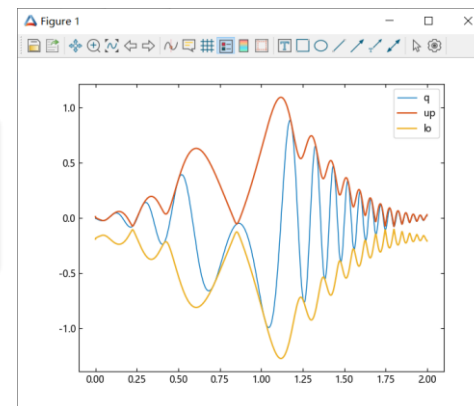
```
x = [4 0 1 6 6  
2 0 2 7 7  
4 0 1 5 5  
2 0 5 6 6  
4 0 1 7 7  
2 0 2 5 5  
4 0 1 6 6  
2 1 5 7 2]  
p = seqperiod(x)
```

```
julia> p  
1×5 Matrix{Int64}:  
2 8 4 3 8
```

示例2：计算信号包络

生成由高斯调制的二次扫频余弦信号。指定 2 kHz 的采样率和 2 秒的信号持续时间。计算扫频余弦信号的上包络和下包络。

```
t = 0:(1/2000):(2-1/2000)  
q = chirp(t, -2, 4, 1 / 2, 6, "quadratic", 100, "convex") .*  
exp.(-4 * (t - 1) .^ 2)  
figure(1)  
plot(t, q)  
up, lo = envelope(q)  
# 上包络、下包络  
hold("on")  
plot(t, up, t, lo; linewidth=1.5)  
legend(["q", "up", "lo"])  
hold("off")  
figure(2)  
envelope(q; plotfig=true)  
# 全包络图绘制
```



信号库是否绘图统一使用
关键字：plotfig

3.1 信号特征提取 – 时域特征

信号处理库提供时域特征提取函数合集

统计量

特征提取和信号标注

脉冲和瞬态指标

函数名	说明
cpnchange	在无变化的情况下计算总剩余误差
energy	计算信号总能量
envelope	信号包络
findpeaks	查找局部最大值
findtroughs	查找局部最小值
meanfreq	平均频率
meansq	计算向量元素的均方
peak2peak	最大到最小差异
peak2rms	峰幅度与RMS的比值
rms	均方根值
rssq	平方根总和
sumsq	序列平方和
seqperiod	计算序列周期

函数名	说明
cusum	使用累计和检测平均值的微小变化
dtw	使用动态时间规整信号之间的距离
finddelay	估计信号之间的延迟
binmask2sigroi	将二进制掩码转换为ROI限制矩阵
extendsigroi	提取感兴趣的信号区域
extractsigroi	将感兴趣的信号区域扩展到左侧和右侧
mergesigroi	合并感兴趣的信号区域
removesigroi	移除感兴趣的信号区域
shortensigroi	从左到右缩短感兴趣的信号区域
sigroi2binmask	将ROI限制矩阵转换为二进制掩码

函数名	说明
dutycycle	脉冲波形占空比
midcross	双电平波形的中间参考电平交叉
pulseperiod	脉冲周期
pulsesep	二值波形脉冲之间的分离
pulsewidth	双电平波形脉冲宽度
statelevels	基于直方图的二值波形状态电平估计
falltime	负向双电平波形转变的下降时间
overshoot	二值波形过渡的超调指标
risetime	正向双电平波形转变的上升时间
settlingtime	双电平波形的稳定时间
slewrate	二值波形的爬升率
undershoot	二值波形跃迁的下冲度量



3.1 信号特征提取 – 时域特征

二.脉冲和瞬态指标

示例3：脉冲指标

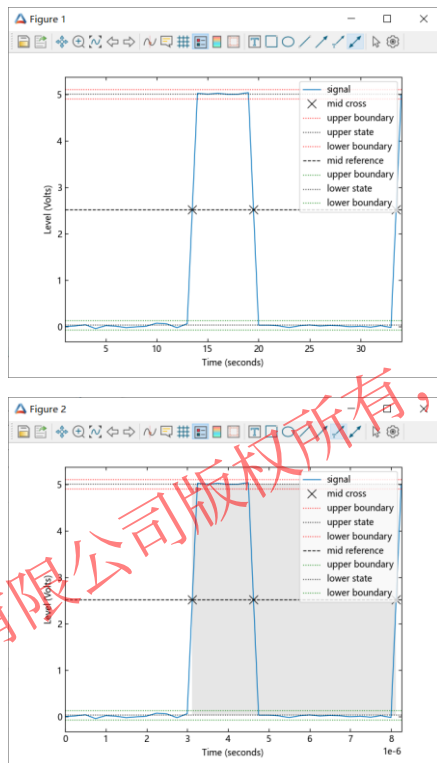
占空比、脉冲周期、脉宽等特征提取

```
using TySignalProcessing:
TySignalProcessing
pkg_dir = pkgdir(TySignalProcessing)
source_path = pkg_dir *
"/examples/Resource/pulseex.mat"
y = TyBase.load(source_path)
x = y["x"]
figure(1)
d, = dutycycle(x; plotfig=true)
# 占空比
figure(2)
p, = pulseperiod(x, t; plotfig=true)
# 脉冲周期
w, = pulsewidth(x, t)
# 脉宽
```

```
julia> d
1-element
Vector{Float64}:
0.3001341713548243
```

```
julia> p
1-element
Vector{Float64}:
5.002996798063269e-6
```

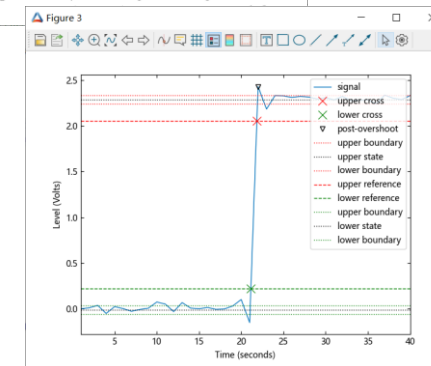
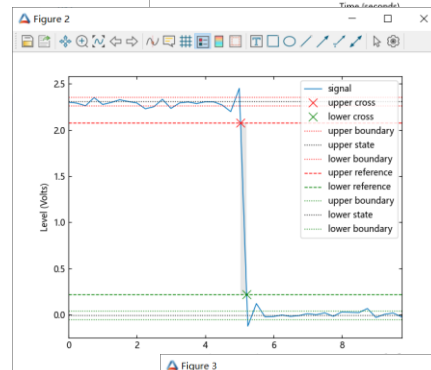
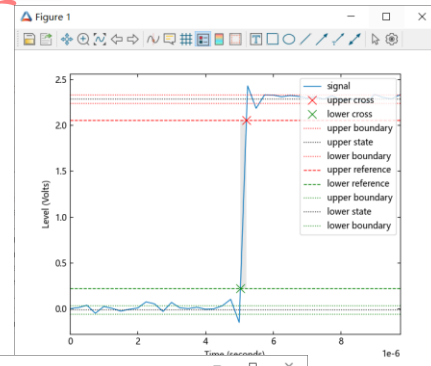
```
julia> w
1-element
Vector{Float64}:
1.5015702982775572e-6
```



示例4：瞬态指标

上升时间、下降时间、超调等特征提取

```
using TySignalProcessing:
TySignalProcessing
pkg_dir = pkgdir(TySignalProcessing)
source_path1 = pkg_dir *
"/examples/Resource/transitionex.m
at"
source_path2 = pkg_dir *
"/examples/Resource/negtransitione
x.mat"
y = TyBase.load(source_path1)
y2 = TyBase.load(source_path2)
x = y["x"]
x2 = y2["x"]
figure(1)
R, = risetime(x, t; plotfig=true)
# 上升时间
figure(2)
F, = falltime(x2, t; plotfig=true)
# 下降时间
figure(3)
O, = overshoot(x; plotfig=true)
# 超调
```



3.2 信号特征提取 – 频域特征

三. 频谱测量

示例5：频率区间总功率的百分比

在白高斯噪声中创建一个由 特定频率正弦波组成的信号，确定指定频率间隔内总功率的百分比

```
rng = MT19937ar(1234)
t = 0:0.001:1-0.001;
x = cos.(2 * pi * 100 * t) + randn(rng, size(t));
# 高斯白噪声中创建100Hz余弦波
x = x[:];
pband = bandpower(x, 1000, [50 150]);
# 50Hz和150Hz频率总功率
ptot = bandpower(x, 1000, [0 500]);
per_power = 100 * (pband / ptot)
# 确定指定频率功率百分比
```

```
julia> pband
0.6303039679351212
```

```
julia> per_power
39.852306177903856
```

示例6：具有和不具有混叠谐波信噪比

生成一个信号，该信号类似于以 2.1 kHz 音调作为输入的弱非线性放大器的输出。该信号以 10 kHz 采样 1 秒。计算并绘制信号的功率谱。使用 $\beta = 38$ 的 Kaiser 窗口进行计算。

```
Fs = 10000;
f = 2100;
t = [0:1/Fs:1;]
rng = MT19937ar(1234)
x = tanh.(sin.(2 * pi * f * t) .+ 0.1) + 0.001 *
randn(rng, length(t))
Sxx, F = periodogram(x, kaiser(length(x), 38),
Fs, nargout=2)
```

使用kaiser窗计算功率谱

figure(1)

```
SNR1, = snr(x, Fs, 7; plotType="power")
```

信噪比，计算不包括最低7次谐波中包含的功率

figure(2)

```
SNR2, = snr(x, Fs, 7; harmType="aliased",
plotType="power")
```

混叠谐波视为信号的

figure(3)

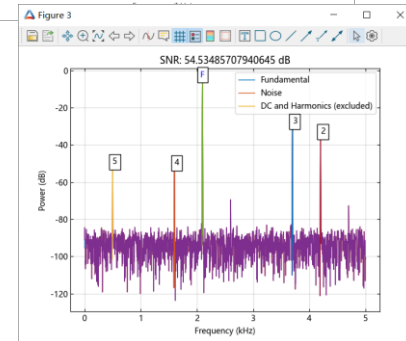
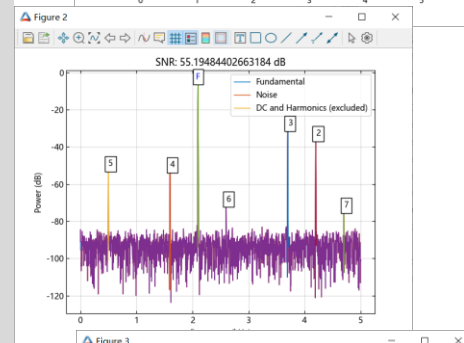
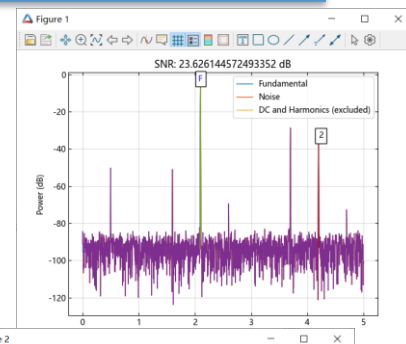
```
SNR3, = snr(x, Fs, 5; harmType="aliased",
plotType="power")
```

计算不包括最低5次谐波中包含的功率

```
julia> SNR1
23.626144572493352
```

```
julia> SNR2
55.19484402663184
```

```
julia> SNR3
54.53485707940645
```



3.2 信号特征提取 – 频域特征

谐波测量函数及如下

函数	说明	用法
sfdr	无杂散动态范围	<pre>r, = sfdr(x) r, = sfdr(x,fs) r, = sfdr(x,fs,msd) r, = sfdr(sxx,f,"power") r, = sfdr(sxx,f,msd,"power") r, spurpow, spurfreq = sfdr(____) sfdr(____; plotfig = Bool)</pre>
sinad	信纳比	<pre>r, = sinad(x) r, = sinad(x,fs) r, = sinad(pxx,f,"psd") r, = sinad(sxx,f,rbw,"power") r, totdistpow = sinad(____) sinad(____;plotfig=Bool)</pre>
snr	信噪比	<pre>r = snr(xi,y) r = snr(x) r = snr(x,fs,n) r = snr(pxx,f,"psd") r = snr(pxx,f,n,"psd") r = snr(sxx,f,rbw, "power") r = snr(sxx,f,rbw,n, "power") r = snr(____; harmType = "aliased") r,noisepow = snr(____)</pre>

thd	总谐波失真	<pre>r, = thd(x) r, = thd(x,fs,n) r, = thd(pxx,f,"psd") r, = thd(pxx,f,n,"psd") r, = thd(sxx,f,rbw,"power") r, = thd(sxx,f,rbw,n,"power") r, = thd(____,harmType = "aliased") r,harpow,harmfreq = thd(____) r,harpow,harmfreq = thd(____;plotfig = Bool)</pre>
toi	三阶截距点	<pre>oip3, = toi(x) oip3, = toi(x,fs) oip3, = toi(pxx,f,"psd") oip3, = toi(sxx,f,rbw,"power") oip3,fundpow,fundfreq,imodpow,imodfreq = toi(____) oip3,fundpow,fundfreq,imodpow,imodfreq = toi(____;plotfig = Bool)</pre>

目录

1. 功能概况

2. 信号预处理

3. 特征提取

4. 滤波器设计与分析

5. 频谱分析

6. 综合示例

4.1 数字和模拟滤波器 – 模拟滤波器

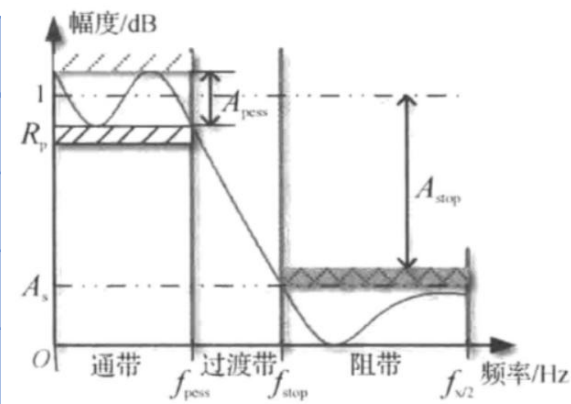
滤波器是一种选频装置，可以滤除干扰噪声或进行频谱分析，按所处理的信号分为模拟滤波器和数字滤波器两种，按所通过信号的频段分为低通、高通、带通、带阻和全通滤波器五种。

信号处理工具箱提供基本滤波器设计方法，对于规格化滤波器用法，可查阅DSP系统工具箱的滤波器内容

一. 模拟滤波器

模拟滤波器分为无源和有源，无源主要由R、L、C构成的，有源主要由运放、电阻、电容构成。信号工具箱提供滤波器的数学模型。

常用模拟滤波器	说明	特性	使用方式
besself	贝塞尔滤波器	最大平坦的群延迟，线性相位响应	<code>__=besself(n,Wo,ftype,"ba")</code>
butter	巴特沃斯滤波器	单调下降幅频特性	<code>__=butter(n, Wn, ftype; otype = "ba")</code>
cheby1	切比雪夫I型滤波器	通带波纹、阻带单调	<code>__=cheby1(n, Rp, Wp,ftype;;otype= "ba")</code>
cheby2	切比雪夫II型滤波器	通带单调、阻带波纹	<code>__=cheby2(n, Rs, Ws,ftype;;otype= "ba")</code>
ellip	椭圆滤波器	过渡带窄，但通带和阻带都是等波纹	<code>__=ellip(n, Rp,Rs, Wp,ftype;otype= "ba")</code>

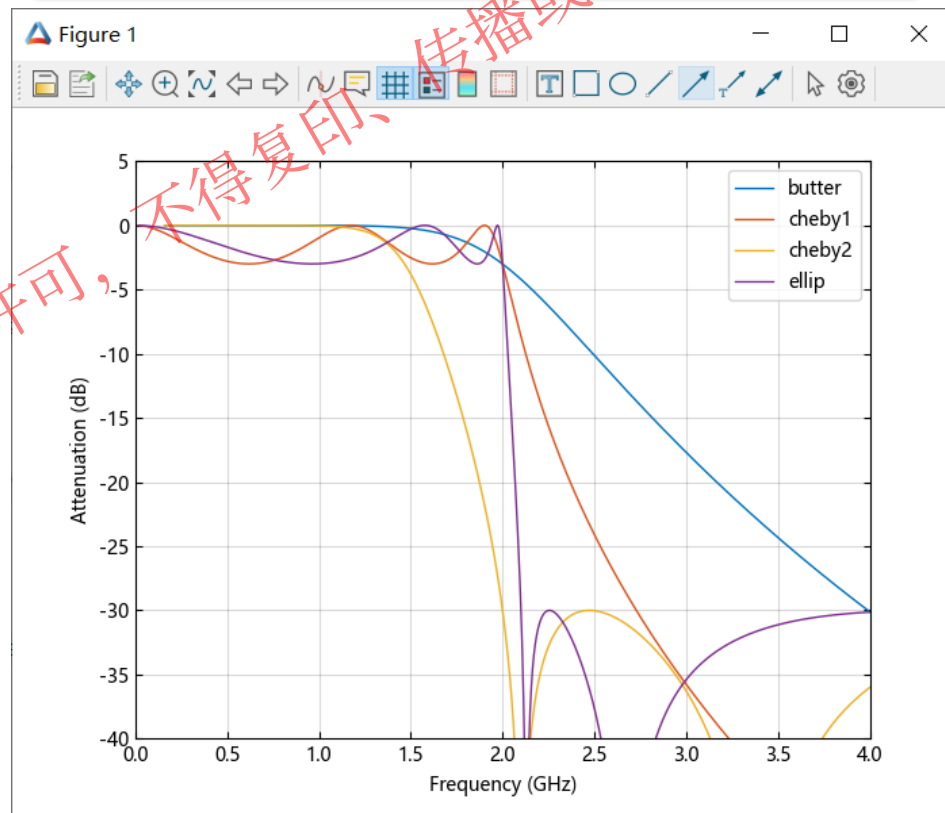


4.1 数字和模拟滤波器 – 模拟滤波器

示例1：模拟低通滤波器幅频响应对比

```
n = 5
f = 2e9
zb,pb,kb = butter(n, 2 * pi * f, "s"; otype="zpk")
# 巴特沃斯滤波器
bb, ab = zp2tf(zb,pb,kb)
hb, wb = freqs(bb, ab, 4096)
z1, p1, k1 = cheby1(n, 3, 2 * pi * f, "s"; otype="zpk")
## 切比雪夫I型, 3dB通带纹波
b1, a1 = zp2tf(z1,p1,k1)
h1, w1 = freqs(b1, a1, 4096)
z2, p2, k2 = cheby2(n, 30, 2 * pi * f, "s"; otype="zpk")
# 切比雪夫II型, 30dB阻带衰减
b2, a2 = zp2tf(z2,p2,k2)
h2, w2 = freqs(b2, a2, 4096)
ze, pe, ke = ellip(n, 3, 30, 2 * pi * f, "s"; otype="zpk")
# 椭圆滤波器, 通带纹波为3dB、阻带衰减为30dB
be, ae, = zp2tf(ze,pe,ke)
he, we = freqs(be, ae, 4096)
plot(wb/(2e9*pi),mag2db.(abs.(hb)))
hold("on")
plot(w1/(2e9*pi),mag2db.(abs.(h1)))
plot(w2/(2e9*pi),mag2db.(abs.(h2)))
plot(we/(2e9*pi),mag2db.(abs.(he)))
axis([0 4 -40 5])
grid("on")
xlabel("Frequency (GHz)")
ylabel("Attenuation (dB)")
legend(["butter", "cheby1", "cheby2", "ellip"])
```

设计一个截止频率为2 GHz的5阶模拟低通滤波器



各类模拟滤波器响应特性与预期一致，通带波纹、阻带衰减
满足输入指标

4.2 数字和模拟滤波器 – 数字滤波器

数字滤波器按所通过信号的频段也可分为低通、高通、带通、带阻和全通滤波器五种，从单位脉冲响应的角度,可以把数字滤波器分为:IIR滤波器（无限长单位冲激响应滤波器）和FIR滤波器（有限长单位冲激响应滤波器）

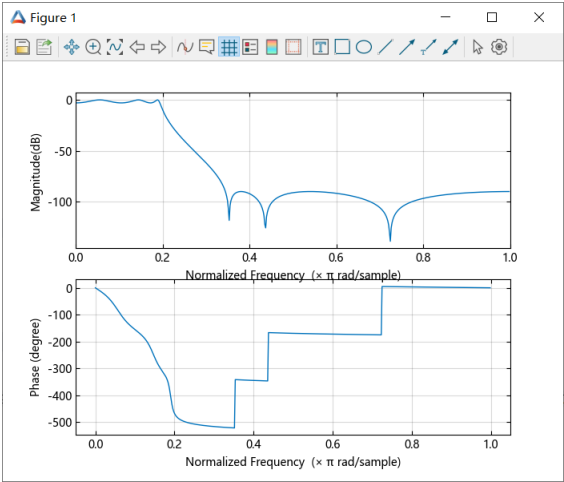
二. IIR数字滤波器

IIR滤波器的设计方法主要有：设计合适的模拟滤波器，再变换值满足预定指标的数字滤波器

函数名	支持数字滤波器	设计方法
besself	不支持	模拟滤波器+双线性变换
butter	<code>___ = butter(___, "z"; otype = __)</code>	模拟滤波器+双线性变换 数字滤波器
cheby1	<code>___ = cheby1(___, "z"; otype = __)</code>	模拟滤波器+双线性变换 数字滤波器
cheby2	<code>___ = cheby2(___, "z"; otype = __)</code>	模拟滤波器+双线性变换 数字滤波器
ellip	<code>___ = ellip(___, "z"; otype = __)</code>	模拟滤波器+双线性变换 数字滤波器

示例2：使用Chebyshev I型模拟滤波器的带通IIR滤波器设计

```
fc = 20
fs = 200
z, p, k = ellip(6, 3, 90, 2 * pi * fc, "lowpass",
"s", otype="zpk")
# 模拟椭圆滤波器设计
b, a = zp2tf(z, p, k)
bd, ad = bilinear(b, a, fs)
# 双线性变换
freqz(bd, ad; plotfig=true)
b1, a1 = ellip(6, 3, 90, 2 * pi * fc, "lowpass",
"s", otype="ba")
# 直接输出ba
bd1, ad1 = bilinear(b1, a1, fs)
freqz(bd1, ad1; plotfig=true)
```



4.2 数字和模拟滤波器 – 数字滤波器

三. FIR数字滤波器

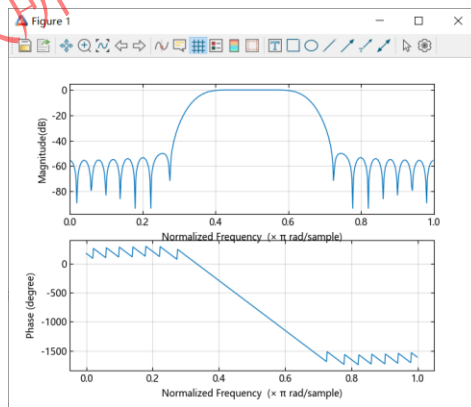
数字滤波器中的FIR滤波器，设计方法主要有：窗函数法、频率抽样法、等波纹逼近法等。

函数名	说明
fir1	基于窗口的FIR滤波器设计
fir2	基于频率采样的FIR滤波器设计
fircls	约束最小二乘FIR多频带滤波器设计
fircls1	约束最小二乘线性相位 FIR 低通和高通滤波器设计
firls	最小二乘线性相位FIR滤波器设计
firpm	Parks-McClellan 最优 FIR 滤波器设计
gaussdesign	高斯FIR脉冲整形滤波器设计
intfilt	内插FIR滤波器设计
maxflat	广义数字巴特沃斯滤波器设计
rcosdesign	升余弦FIR脉冲整形滤波器设计

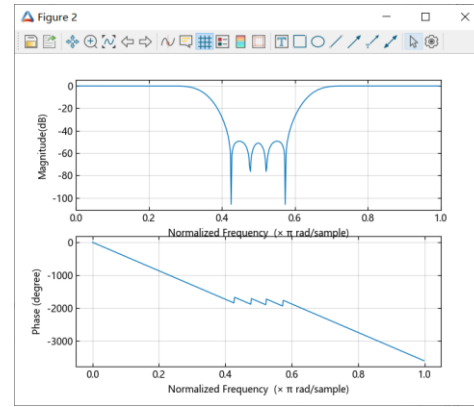
示例3：基于窗函数的FIR滤波器设计

目标：设计一个通带（阻带）为 $0.35\pi \leq \omega \leq 0.65\pi$ rad/sample 的 48 阶 FIR 带通滤波器。可
视化其幅度和相位响应。

```
b = fir1(48, [0.35 0.65], "bandpass")  
freqz(b, [1], 512, plotfig = true)
```



```
b = fir1(48, [0.35 0.65], "bandstop")  
freqz(b, [1], 512, plotfig = true)
```



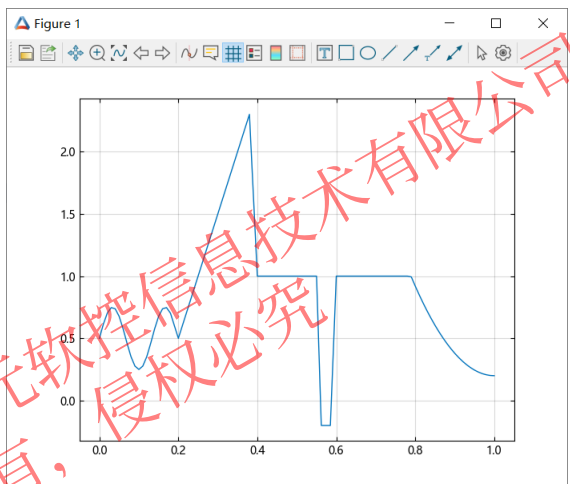
`fir1(n, Wn, ftype, window)`
ftype: " lowpass " | " highpass " | " bandpass " | " bandstop "
window: 指定窗，默认汉宁窗

4.2 数字和模拟滤波器 – 数字滤波器

示例4：基于频率采样的 FIR 滤波器设计

设计具有任意幅频响应的FIR滤波器

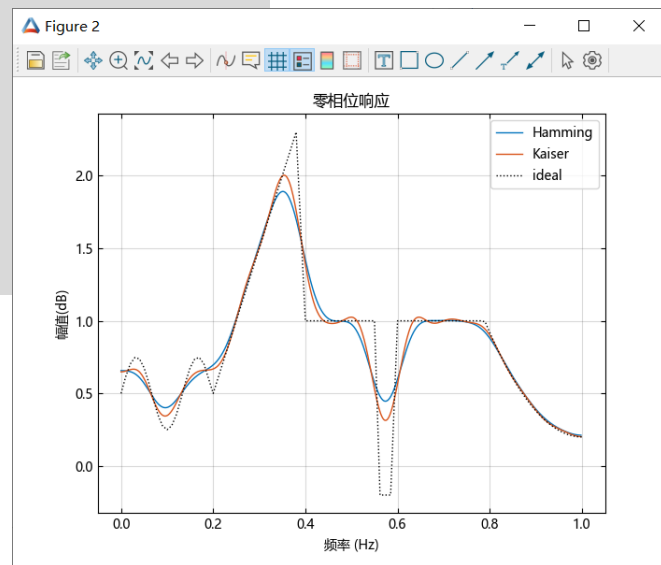
```
F1 = [0:0.01:0.18...]
A1 = 0.5 .+ sin.(2 * pi * 7.5 * F1) / 4
F2 = [0.2, 0.38, 0.4, 0.55, 0.562, 0.585, 0.6, 0.78]
A2 = [0.5, 2.3, 1, 1, -0.2, -0.2, 1, 1]
F3 = [0.79:0.01:1...]
A3 = 0.2 .+ 18 * ((1) .- F3) .^ 2
FreqVect = [F1; F2; F3]
AmplVect = [A1; A2; A3]
figure(1)
plot(FreqVect, AmplVect)
grid("on")
```



目标幅频响应

```
N=50
ham = fir2(N, FreqVect, AmplVect)
# 使用汉明窗设计滤波器。指定滤波器阶数为50
kai = fir2(N, FreqVect, AmplVect, kaiser(N + 1, 3))
# 使用形状参数为3的Kaiser窗口重复计算
hr_ham, w = freqz(ham, [1])
hr_kai, = freqz(kai, [1])
figure(2)
plot(w / pi, abs(hr_ham))
grid()
hold("on")
plot(w / pi, abs(hr_kai))
plot(FreqVect, AmplVect, "k:")
ylabel("幅值(dB)")
xlabel("频率 (Hz)")
legend(["Hamming", "Kaiser", "ideal"])
title("零相位响应")
hold("off")
```

根据实际需要去指定合适的滤波器阶数及窗



4.3 数字和模拟滤波器 – 滤波器分析

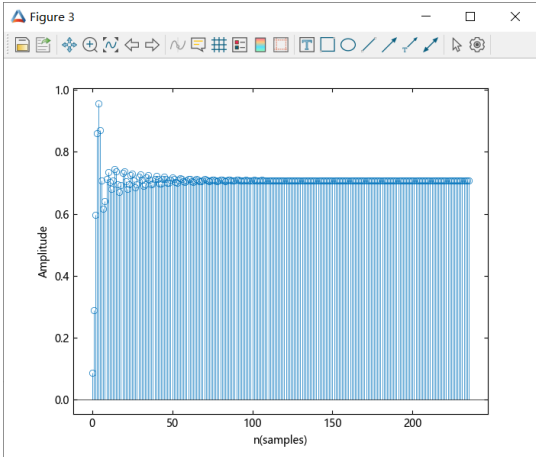
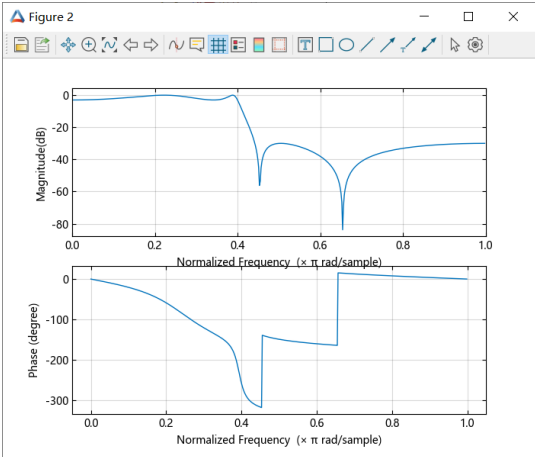
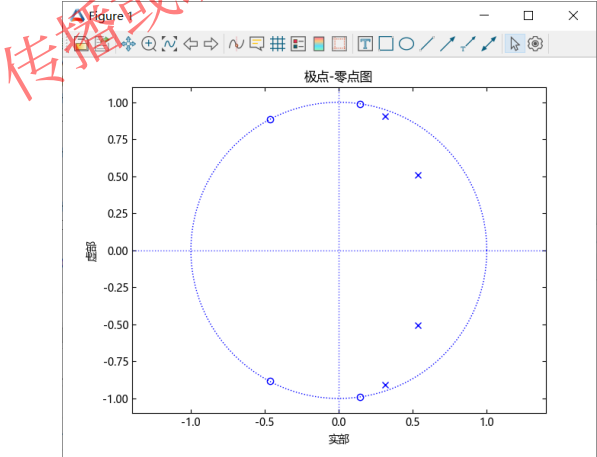
四. 滤波器分析

提供时域、频域响应等功能

函数名	说明
freqz	数字滤波器的频率响应
findfreqs	查找用于计算模拟滤波器响应的频率数组
fvtool	将滤波器可视化
grpdelay	平均滤波器延迟（组延迟）
phasez	数字滤波器的相位延迟
phasedelay	数字滤波器的相位延迟
zerophase	数字滤波器的零相位响应
zplane	离散系统的零极点图
impz	数字滤波器的脉冲响应
stepz	数字滤波器的阶跃响应
freq	模拟滤波器的频率响应

示例5：椭圆低通滤波器的极点和零点

```
z, p, k = ellip(4, 3, 30, 200 / 500, "lowpass";
otype="zpk")
zplane(z, p)
# 零极点分布
b, a = zp2tf(z, p, k)
freqz(b, a, plotfig=true)
# 频率响应
h, t = stepz(vec(b), vec(a))
# 阶跃响应
figure()
stem(t, h)
xlabel("n(samples)")
ylabel("Amplitude")
```



目录

1. 功能概况

2. 信号预处理

3. 特征提取

4. 滤波器设计与分析

5. 频谱分析

6. 综合示例

5.1 频谱分析- 经典谱估计

信号的频谱分析是研究信号特性的重要手段之一，通常是求其功率谱来进行频谱分析。功率谱反映了随机信号各频率成份功率能量的分布情况，可以提示信号中隐含的周期性及靠得很近的谱峰等有用信息，在许多领域都发挥了重要作用。

实际应用中的平稳随机信号通常是有限长的，只能根据有限长信号估计原信号的真实功率谱。因此，功率谱估计就是基于有限的数据寻找信号、随机过程或系统的频率成分。

谱估计方法可以分成两大类：经典估计方法和现代估计方法。

通常将基于相关函数的傅里叶变换的估计方法称为经典功率谱估计，而将参数模型估计方法和基于相关矩阵特征值分解的信号频率估计方法，称为现代功率谱估计方法。

一.经典谱估计

经典谱估计方法类型	函数	特点
基于周期图的功率谱估计	periodogram	优点：针对周期信号的频谱估计能够提供较高的频率分辨率，且容易理解。 缺点：不适用于非周期信号，同时对于周期性不强的信号会导致估计结果不准。 适用场景：适用于周期性强、频率固定的信号的频谱估计。
基于welch的功率谱估计	pwelch	优点：不依赖于信号的周期性，且采用重叠平均法进行频谱估计，能够克服干扰和噪声。 缺点：需要根据信号的特性调整窗口长度和重叠比例，保证估计效果。 适用场景：适用于样本数较少或信号具有不稳定性质的频谱估计。
基于多窗口的功率谱估计	pmtm	优点：MTM能够提供高分辨力的频谱估计结果，相比于传统的FFT方法具有更好的误差性能。 缺点：需要调整多个参数，对于信号的长度要求比较高，处理时间较长。 适用场景：对于长时域数据的频谱分析，MTM能够提供更加准确的频谱图像。

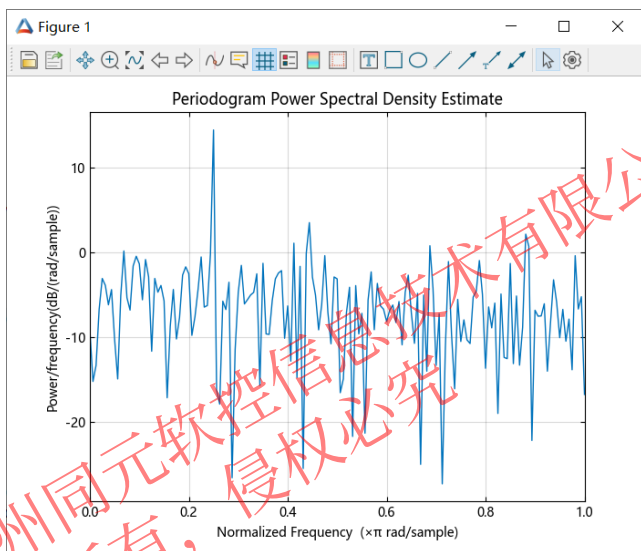


5.1 频谱分析—经典谱估计

获取输入信号的周期图，该信号由角频率为 $\pi/4$ rad/sample 的离散时间正弦波和加性 $N(0, 1)$ 白噪声组成。使用与信号长度相等的 DFT 长度。创建一个角频率为 $\pi/4$ rad/sample 的正弦波，带有加性 $N(0, 1)$ 白噪声。该信号的长度为 320 个样本。

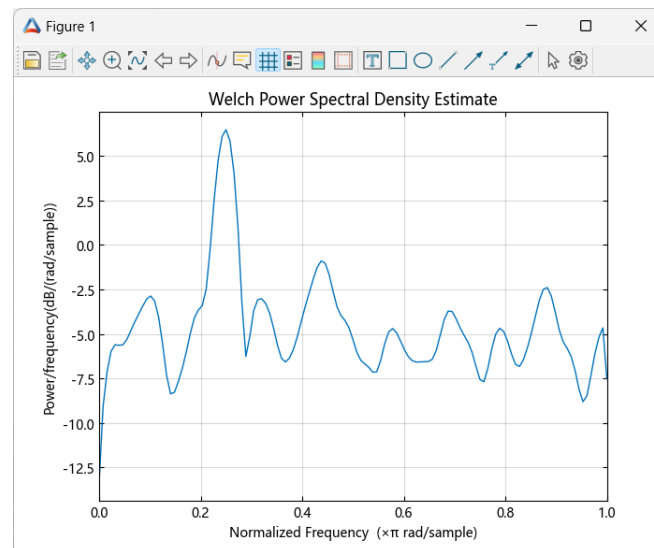
示例1：基于周期图的功率谱估计

```
rng = MT19937ar(1234)
n = 0:319;
x = cos.(pi / 4 * n)' + randn(rng, 1, size(n, 1));
nfft = length(x);
pxx = periodogram(x, [], nfft)
# 功率谱密度估计
periodogram(x, [], nfft; plotfig=true)
# 绘制功率谱密度估计曲线
```



示例2：基于welch方法的功率谱估计

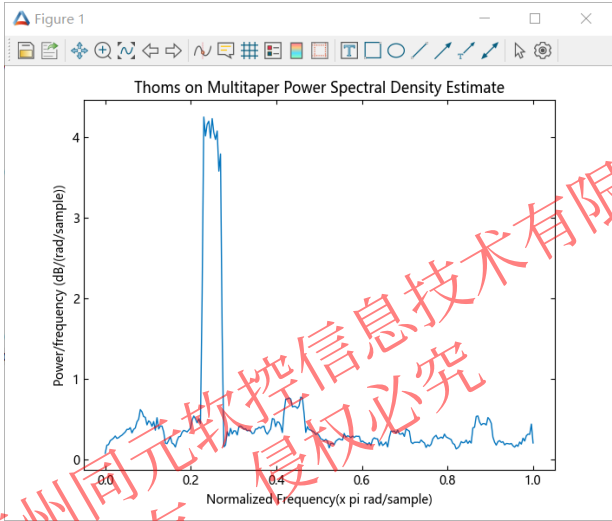
```
rng = MT19937ar(1234)
n = 0:319;
x = cos.(pi / 4 * n)' + randn(rng, (1, size(n, 1)))
pxx, w = pwelch(x; nargout=2)
# 功率谱密度估计
pwelch(x; nargout=0, plotfig=true)
# 绘制功率谱密度估计曲线
```



5.1 频谱分析- 经典谱估计

示例3：基于多窗口的功率谱密度估计

```
rng = MT19937ar(1234)
n = 0:319
x = cos.(pi / 4 * n) + randn(rng, size(n))
a, b = pmtm(x)
# 功率谱密度估计
figure()
plot(b/pi, a)
xlabel("Normalized Frequency(x pi rad/sample)")
ylabel("Power/frequency (dB/(rad/sample))")
title("Thoms on Multitaper Power Spectral Density Estimate")
```



经典谱估计方法	用法
periodogram	<pre>pxx = periodogram(x) pxx = periodogram(x, window) pxx = periodogram(x, window, nfft) pxx, w = periodogram(___, nargout = 2) pxx, f = periodogram(___, fs; nargout=2) pxx, w = periodogram(x, window, w; nargout = 2) pxx, f = periodogram(x, window, f, fs; nargout=2) ___ = periodogram(x, window, ___, freqrange; nargout = nargout) ___, pxxc = periodogram(___, "ConfidenceLevel", probability; nargout = nargout)</pre>
pwelch	<pre>pxx = pwelch(x) pxx = pwelch(x, window) pxx = pwelch(x, window, noverlap) pxx = pwelch(x, window, noverlap, nfft) pxx, w = pwelch(___; nargout = 2) pxx, f = pwelch(___, fs; nargout = 2) pxx, w = pwelch(___; nargout = 2) pxx, f = pwelch(___, fs; nargout = 2) ___ = pwelch(x, window, ___, freqrange; nargout = nargout) ___ = pwelch(x, window, ___, trace; nargout = nargout) ___, pxxc = pwelch(___, "ConfidenceLevel", probability; nargout = 3) ___ = pwelch(___, spectrumtype; nargout = nargout) pwelch(___; plotfig = true)</pre>
pmtm	<pre>pxx, w = pmtm(x) pxx, w = pmtm(x, nw) pxx, w = pmtm(x, nw, nfft)</pre>



5.2 频谱分析 – 现代谱估计

现代功率谱估计是非线性估计方法，主要是参数化方法：

- 参数估计方法性能依赖于参数模型，最终可获得方差小、分辨率高的谱估计，但所用模型必须适合于所分析信号，否则谱估计将是错误或者不准确的。有AR模型，MA模型，ARMA模型等；
- 非参数模型谱估计有最小方差法和MUSIC法等。

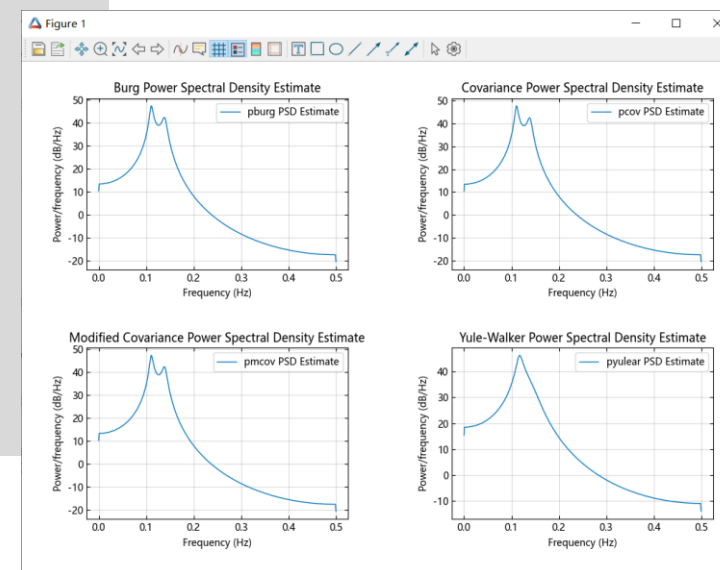
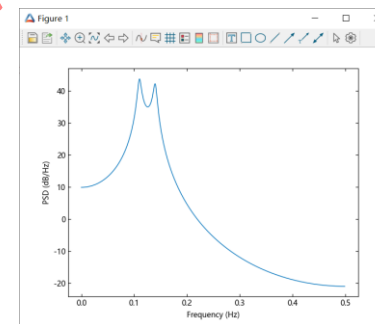
二. 参数化谱估计

示例4：自回归模型的参数化功率谱密度估计

```
A = [1, -2.7607, 3.8106, -2.6535, 0.9238]
H, F = freqz(1, A, [], 1)
figure(1)
plot(F, 20 * log10(abs.(H)))
xlabel("Frequency (Hz)")
ylabel("PSD (dB/Hz)")
# 创建 AR(4) 系统功能。
# 获取频率响应并绘制系统 PSD。
rng = MT19937ar(1234)
x = randn(rng, 1000)
y, = filter1(1, A, x)
```

```
figure(2)
subplot(2, 2, 1)
Pxx1, = pburg(y, 4, 1024; fs=1, plotfig=true)
# Burg方法
legend("pburg PSD Estimate")
subplot(2, 2, 2)
Pxx2, = pcov(y, 4, 1024; fs=1, plotfig=true)
# 协方差方法
legend("pcov PSD Estimate")
subplot(2, 2, 3)
Pxx3, = pmcov(y, 4, 1024; fs=1, plotfig=true)
# 修正协方差方法
legend("pmcov PSD Estimate")
subplot(2, 2, 4)
Pxx4, = pyulear(y, 4, 1024; fs=1, plotfig=true)
# Yule-Walker方法
legend("pyulear PSD Estimate")
```

Burg、Yule-Walker、协方差和修正协方差方法均使用基于自回归（AR）模型的参数化方法来估计频谱。



5.2 频谱分析 – 现代谱估计

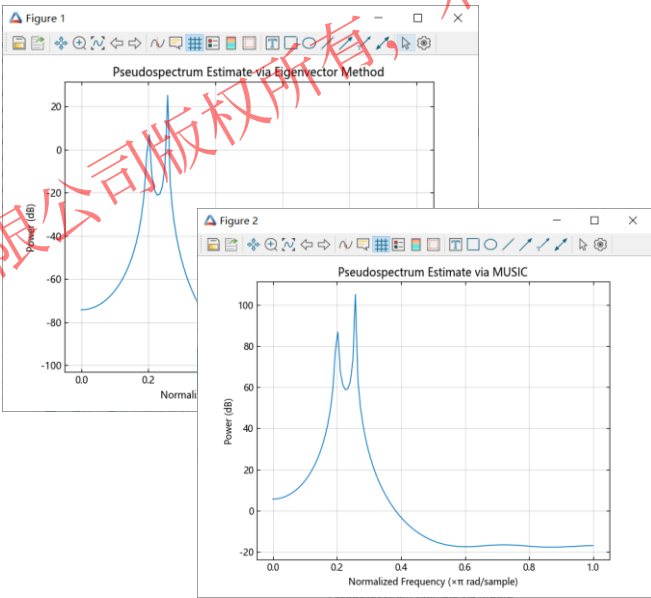
三. 子空间法频谱估计

高分辨率伪频谱估计，提供基于子空间方法的多信号分类 (MUSIC) 算法和特征向量方法。

示例5：正弦波叠加后的伪谱

假设信号子空间中存在两个实正弦分量。在这种情况下，信号子空间的维数是 4，因为每个实正弦曲线是两个复指数的和。

```
n = 0:199;
rng = MersenneTwister(1234)
x = cos.(0.257.* pi.* n) .+ sin.(0.2.* pi.* n) .+
0.01 * randn(rng, length(n))
Pxx, w, = pmusic(x, 4; plotfig=true)
p, f = peig(x, 4; plotfig=true)
# 基于特征向量法的伪谱估计
figure(2)
Pxx, w, = pmusic(x, 4; plotfig=true)
# 基于MUSIC算法的伪谱估计
```



类型	函数	说明
经典频谱估计	periodogram	周期图功率谱密度估计
	pwelch	Welch 的功率谱密度估计
	pmtm	多窗口功率谱密度估计
现代频谱估计	pburg	自回归功率谱密度估计 - Burg法
	pyulear	自回归功率谱密度估计 - Yule-Walker法
	pcov	自回归功率谱密度估计 - 协方差法
	pmcov	自回归功率谱密度估计 - 修正协方差法
	peig	使用特征向量法的伪谱估计
	pmusic	使用MUSIC算法的伪谱估计

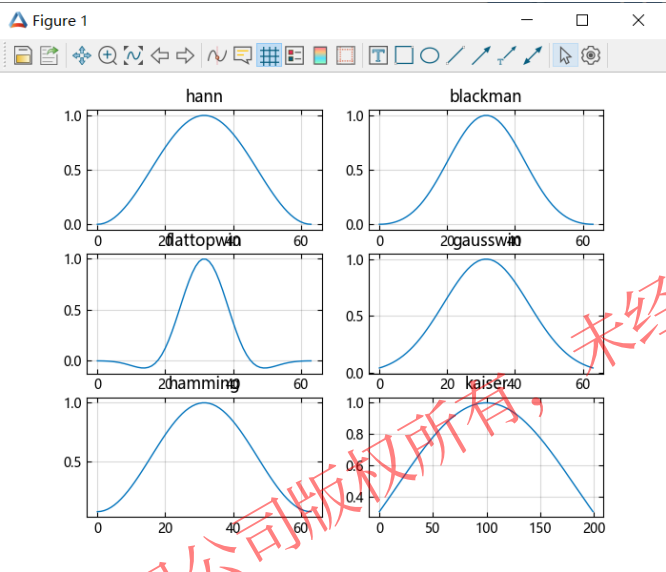


5.3 频谱分析 – 窗函数

四. 窗函数

示例6：典型窗函数

```
y_hann = hann(64)
# 汉宁窗
y_blackman = blackman(64)
# 布莱克曼窗
y_flattopwin = flattopwin(64)
# 平顶窗
y_gausswin = gausswin(64)
# 高斯窗
y_hamming = hamming(64)
# 汉明窗
y_kaiser = kaiser(200, 2.5)
# 凯撒窗
figure(1)
subplot(3,2,1);plot(y_hann);grid("on");title("hann");
subplot(3,2,2);plot(y_blackman);grid("on");title("blackman");
subplot(3,2,3);plot(y_flattopwin);grid("on");title("flattopwin");
subplot(3,2,4);plot(y_gausswin);grid("on");title("gausswin");
subplot(3,2,5);plot(y_hamming);grid("on");title("hamming");
subplot(3,2,6);plot(y_kaiser);grid("on");title("kaiser");
```



函数名	说明
barthannwin	改良的Bartlett-Hann窗
bartlett	巴特利特窗口
blackman	布莱克曼窗口
blackmanharris	最小四项Blackman-Harris窗
bohmanwin	Bohman窗口
cheb	返回在x点处计算的n阶切比雪夫多项式的值
chebwin	切比雪夫窗
cosine	余弦窗
enbw	等效噪声带宽
flattopwin	平顶窗
gausswin	高斯窗
hamming	汉明窗
hann	汉宁窗
kaiser	凯撒窗
lanczos	lanczos窗
nuttallwin	Nuttall定义的最小4项Blackman-Harris窗
parzenwin	帕尔森（德拉瓦勒普桑）窗口
rectwin	矩形窗
taylorwin	泰勒窗
triang	三角窗
tukeywin	Tukey（锥形余弦）窗口



目录

1. 功能概况

2. 信号预处理

3. 特征提取

4. 滤波器设计与分析

5. 频谱分析

6. 综合示例

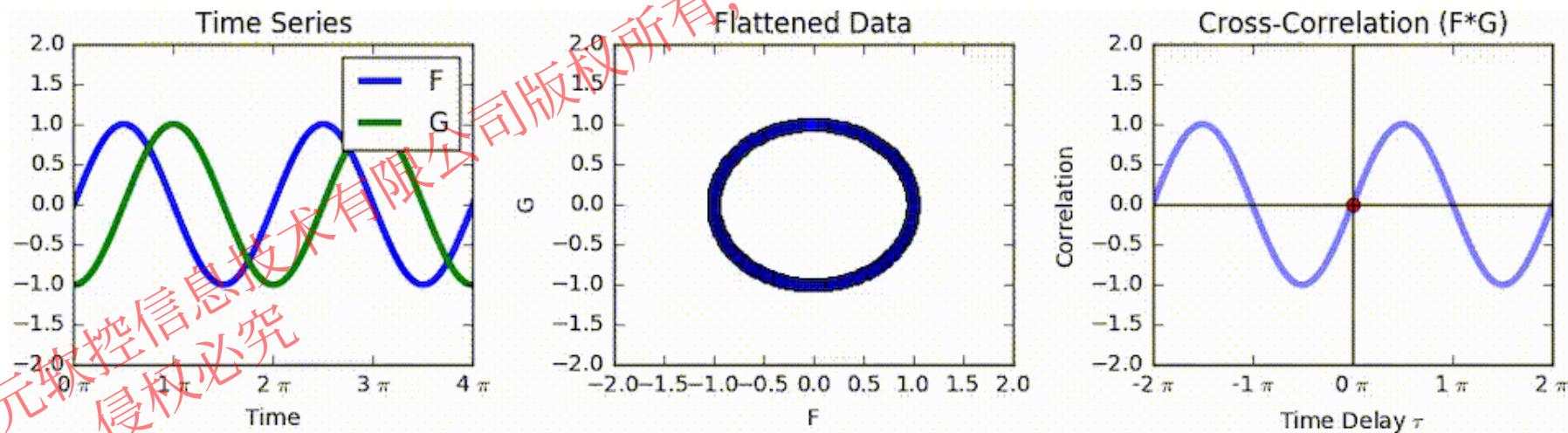
6 综合示例 – 声源定位

在日常生活中，我们的耳朵会听到各种声音并进行识别定位，即所谓的“听声辨位”。

声源定位算法主要有以下几大类：

- 1、波束成形法 (Beamforming) 。
- 2、声全息法 (Holography)
- 3、时差法 (Time Difference of Arrival)

TDOA方法主要核心是计算声源到达各个麦克风的时间差，以此得到声源、麦克风相对距离用以位置估计。其中时差法由互相关或广义互相关算法计算得到。本文以时差法为例作为信号工具箱的实践应用。



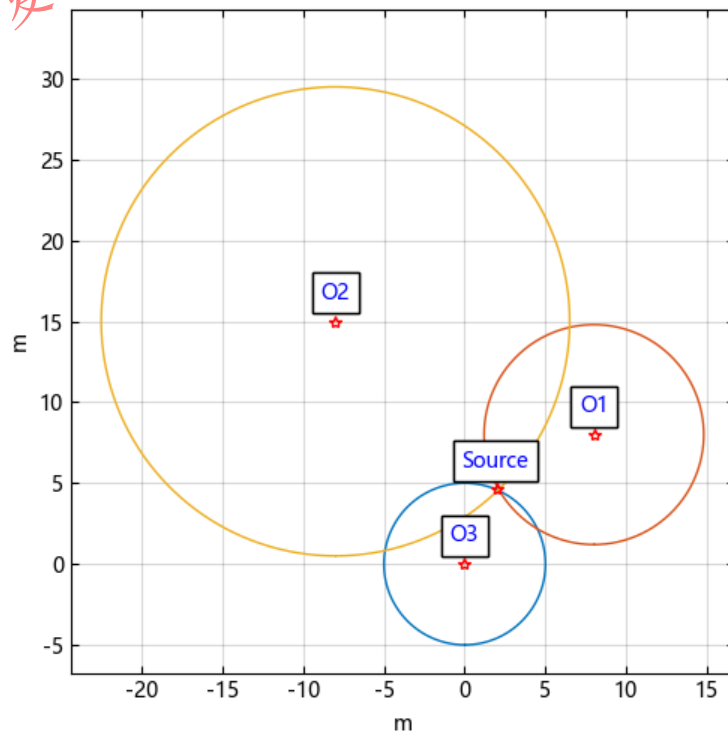
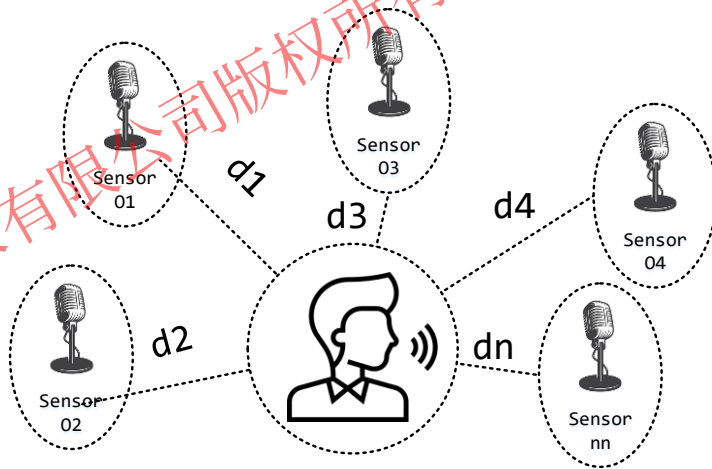
6 综合示例 – 声源定位

时差定位法——基于相对距离的多传感器定位方法

仅考虑二维平面空间，采用基于时延的声源定位方法，布置N个传感器进行坐标解算。

未知声源坐标 $[X_t, Y_t]$ ，已知传感器 $[X_{O1}, Y_{O1}]$ 、 $[X_{O2}, Y_{O2}]$ 、 $[X_{O3}, Y_{O3}]$ 、.....，传感器间时延 ΔT_{13} 、 ΔT_{23} 、.....，为实现声源位置解算，需要至少布置三个及以上传感器，构成有效方程个数大于未知数个数的超定方程组，并采用优化方法得到最优解。本文针对仅布置三个麦克风通过迭代求解非线性方程数组的数值解。

$$\begin{cases} (X_t - X_{O1})^2 + (Y_t - Y_{O1})^2 = d1^2 \\ (X_t - X_{O2})^2 + (Y_t - Y_{O2})^2 = d2^2 \\ (X_t - X_{O3})^2 + (Y_t - Y_{O3})^2 = d3^2 \\ \dots\dots \\ d3 - d1 = \Delta T_{13} * V_{sound} \\ d3 - d2 = \Delta T_{23} * V_{sound} \\ \dots\dots \end{cases}$$

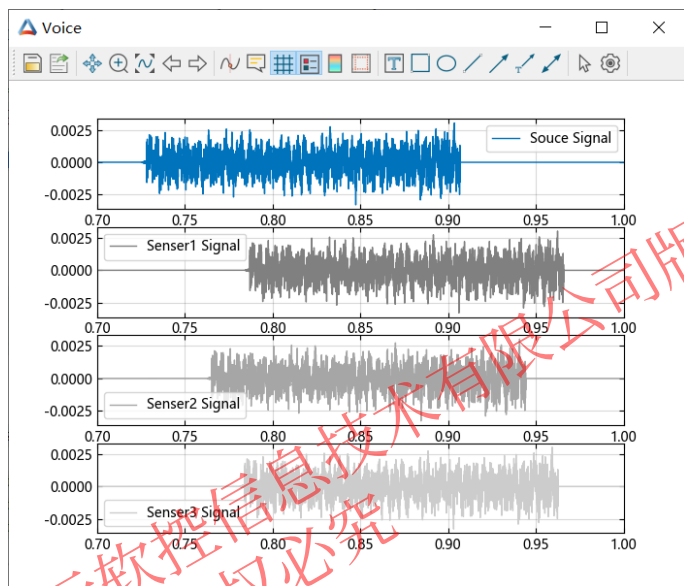


6 综合示例 – 声源定位

信号检测阶段的主要目标是计算不同麦克风对同一声源信号的时延，需要依次完成采样、互相关分析、时延估计操作。

■ 信号采样

模拟声源在安静室内发出持续时间约0.2秒的声波，三组麦克风持续监听，并完成音频信号的采样记录，采样率越高，信号检测算法越精确，本文采用采样频率为常规44100Hz。



注意：需要保证当前目录下有RES-example.wav文件

```
using WAV
y, fs = wavread("RES-example.wav")
Full_sig = vec(y)
ptime = [1/fs:1/fs:length(y)/fs;]
ps = [12, 4] # 声源实际坐标
po1 = [0, 20] # 麦克风1实际坐标
po2 = [0, 0] # 麦克风2实际坐标
po3 = [30, 10] # 麦克风3实际坐标
vsound = 340
o1t = sqrt(sum((po1 - ps).^2)) / vsound
o2t = sqrt(sum((po2 - ps).^2)) / vsound
o3t = sqrt(sum((po3 - ps).^2)) / vsound
n1 = floor(int,o1t * fs)
n2 = floor(int,o2t * fs)
n3 = floor(int,o3t * fs)
mFull_sig = [zeros(30000); Full_sig[1:10000]; zeros(80000)]
mptime = ptime[1:120000]
ro1 = [zeros(n1); mFull_sig[1:end-n1]]
ro2 = [zeros(n2); mFull_sig[1:end-n2]]
ro3 = [zeros(n3); mFull_sig[1:end-n3]]
figure("Voice")
subplot(4, 1, 1);plot(mptime, mFull_sig)
legend(["Souce Signal"]);xlim([0.7, 1.0]);grid("on")
subplot(4, 1, 2);plot(mptime, ro1, color=[0.50, 0.50, 0.50])
grid("on");xlim([0.7, 1.0]);legend(["Senser1 Signal"])
subplot(4, 1, 3)
plot(mptime, ro2, color=[0.65, 0.65, 0.65]);
grid("on");legend(["Senser2 Signal"]);xlim([0.7, 1.0])
subplot(4, 1, 4)
plot(mptime, ro3, color=[0.80, 0.80, 0.80])
grid("on");legend(["Senser3 Signal"]);xlim([0.7, 1.0])
```

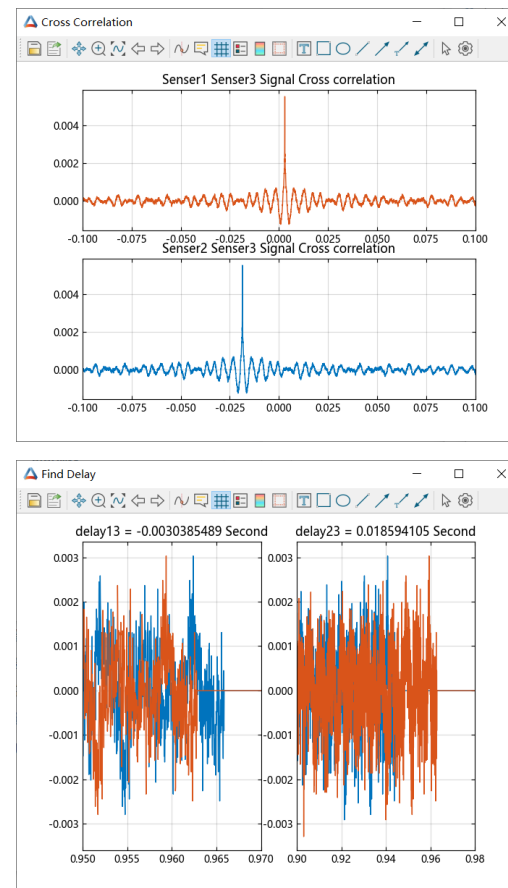
6 综合示例 – 声源定位

■ 互相关与时延分析

选择传感器3作为基准信号，用于计算接收信号的时延，基于相关性理论，采用核心函数xcorr对信号序列计算互相关及自相关，得到相关性发布曲线。

```
# 互相关计算
xCorr33, lags33 = xcorr(ro3)
xCorr13, lags13 = xcorr(ro1, ro3)
xCorr23, lags23 = xcorr(ro2, ro3)
# 绘图, 相关性
figure("Cross Correlation")
subplot(2, 1, 1)
plot(lags13 / fs, xCorr13, color=[0.85, 0.33, 0.10])
xlim([-0.1, 0.1])
grid("on")
title("Senser1 Senser3 Signal Cross correlation")
subplot(2, 1, 2)
plot(lags23 / fs, xCorr23)
xlim([-0.1, 0.1])
grid("on")
title("Senser2 Senser3 Signal Cross correlation")
```

```
# 检测
_, l33 = findmax(abs(xCorr33))
_, l13 = findmax(abs(xCorr13))
_, l23 = findmax(abs(xCorr23))
dealy13 = (l33 - l13) / fs
dealy23 = (l33 - l23) / fs
figure("Find Delay")
subplot(1, 2, 1)
plot(mptime, ro1)
hold("on")
plot(mptime, ro3)
grid("on")
xlim([0.95, 0.97])
title("delay13 = $dealy13 Second")
subplot(1, 2, 2)
plot(mptime, ro2)
hold("on")
plot(mptime, ro3)
grid("on")
xlim([0.9, 0.98])
title("delay23 = $dealy23 Second")
```



6 综合示例 – 声源定位

■ 声源定位

针对布置有三组麦克风的室内环境，描述声源平面坐标的方程组如下：

$$\begin{cases} (X_t - X_{o1})^2 + (Y_t - Y_{o1})^2 = (d3 - \Delta T_{13} * V_{sound})^2 \\ (X_t - X_{o2})^2 + (Y_t - Y_{o2})^2 = (d3 - \Delta T_{23} * V_{sound})^2 \\ (X_t - X_{o3})^2 + (Y_t - Y_{o3})^2 = d3^2 \end{cases}$$

其中，声源坐标为 $[X_t, Y_t]$ ，已知传感器坐标 $[X_{O1}, Y_{O1}]$ 、 $[X_{O2}, Y_{O2}]$ 、 $[X_{O3}, Y_{O3}]$ ，传感器间时延差为 ΔT_{13} 、 ΔT_{23} ， $d3$ 为传感器 $O3$ 到声源的距离。

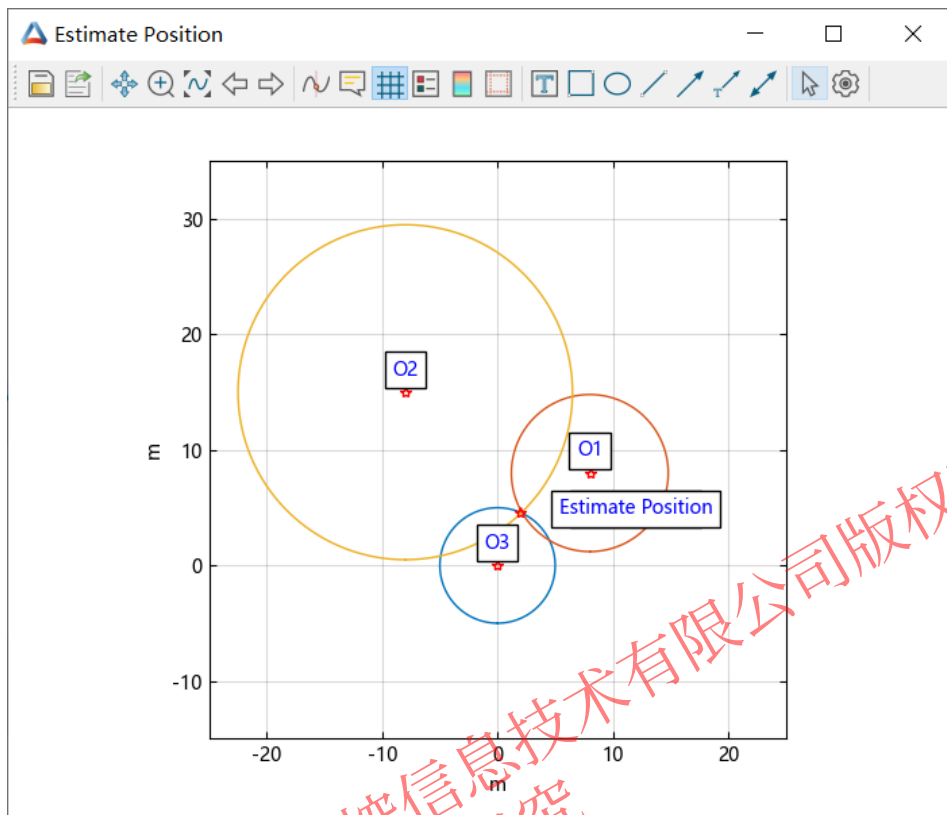
考虑麦克风信号采样及互相关算法存在一定误差，需采用优化方法需求方程解，给定声源坐标迭代初值，并寻求非线性方程最优数值解。

fsolve基于最小二乘求得最优数值解，与实际位置 $[12, 4]$ 在容许误差范围内基本吻合

```
function fun(x)
    xo1, yo1 = [0, 20]
    xo2, yo2 = [0, 0]
    xo3, yo3 = [30, 10]
    delay13 = -0.0030385489f0
    delay23 = 0.018594105f0
    vsound = 340
    out = [(x[1] - xo1)^2 + (x[2] - yo1)^2 - (x[3] - delay13 * vsound)^2,
           (x[1] - xo2)^2 + (x[2] - yo2)^2 - (x[3] - delay23 * vsound)^2,
           (x[1] - xo3)^2 + (x[2] - yo3)^2 - x[3]^2]
    return out
end
x0 = [5, 5, 10]
result = fsolve(fun, x0)
xe = result[1]
```

```
julia> result.x
3-element Vector{Float64}:
 12.002910281400293
  3.995315616123038
 18.972387116263963
```


6 综合示例 – 声源定位



估计位置

```
figure("Estimate Position")
title("Indoor")
u = [-pi:0.01:pi];
mt1x = 0 .+ sin.(u) * 5
mt1y = 0 .+ cos.(u) * 5
plot(mt1x, mt1y)
hold("on")
mt2x = 8 .+ sin.(u) * 6.8
mt2y = 8 .+ cos.(u) * 6.8
plot(mt2x, mt2y)
mt3x = -8 .+ sin.(u) * 14.5
mt3y = 15 .+ cos.(u) * 14.5
plot(mt3x, mt3y)
text(ps[1], ps[2], "True Position", va="bottom", ha="center", color="blue",
bbox=Dict("edgecolor" => "k", "facecolor" => "w"))
text(xe[1], xe[2], "Estimate Position", va="bottom", ha="center", color="blue",
bbox=Dict("edgecolor" => "k", "facecolor" => "w"))
grid("on")
xlabel("m")
ylabel("m")
xlim([-25, 25])
ylim([-15, 35])
axis("square")
text(8, 9, "O1", va="bottom", ha="center", color="blue", bbox=Dict("edgecolor"
=> "k", "facecolor" => "w"))
text(-8, 16, "O2", va="bottom", ha="center", color="blue",
bbox=Dict("edgecolor" => "k", "facecolor" => "w"))
text(0, 1, "O3", va="bottom", ha="center", color="blue", bbox=Dict("edgecolor"
=> "k", "facecolor" => "w"))
plot(0, 0, "r*", 8, 8, "r*", -8, 15, "r*")
plot(2, 4.62757, "r*")
```


建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

Thanks !



@苏州同元软控信息技术有限公司版权所有，侵权必究